Problem List | Submit | 0 | 02:25:12 | Premium

Description | Editorial | Solutions | Submi

## 20. Valid Parentheses

Solved

Easy | Topics | Companies | Hint

Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

**Example 1:**

28K | 664 | 534 Onlin

Code

Java | Auto

```java
class Solution {
    public boolean isValid(String s) {
        Stack<Character> sk = new Stack<>();
        for(int i=0;i<s.length();i++){
            char ch = s.charAt(i);
            //Opening
            if(ch == '(' || ch=='{' || ch=='['){
                sk.push(ch);
            }
```

Saved | Ln 1, Col 1

Testcase | Test Result | Accepted

All Submissions

Accepted 102 / 102 testcases passed

rutuja_likhar2003 submitted at Apr 20, 2026 21:40

Analysis | Solution

Runtime: 3 ms | Beats 87.83%

Memory: 43.30 MB | Beats 60.38%

Problem List | Submit | 0 | 02:25:12 | Premium

Description | Editorial | Solutions | Submissions

## 1047. Remove All Adjacent Duplicates In String

Solved

Easy | Topics | Companies | Hint

You are given a string `s` consisting of lowercase English letters. A **duplicate removal** consists of choosing two **adjacent** and **equal** letters and removing them.

We repeatedly make **duplicate removals** on `s` until we no longer can.

Return *the final string after all such duplicate removals have been made.* It can be proven that the answer is **unique**.

**Example 1:**

```
Input: s = "abbaca"
Output: "ca"
Explanation:
```

7.1K | 140 | 38 Online

Code

Java | Auto

```java
class Solution {
    public String removeDuplicates(String s) {
        Stack<Character> st = new Stack<>();
        StringBuilder str =new StringBuilder();
        for(int i=0;i<s.length();i++){
            char ch = s.charAt(i);
            if(!st.isEmpty() && ch == st.peek()){
                st.pop();
            }
            else{
                st.push(ch);
```

Saved | Ln 1, Col 1

Testcase | Test Result | Accepted

All Submissions

Accepted 106 / 106 testcases passed

rutuja_likhar2003 submitted at Apr 20, 2026 21:47

Analysis | Solution

Runtime: 37 ms | Beats 26.46%

Memory: 47.25 MB | Beats 40.95%

Problem List | Submit | 0 | 02:25:12 | Premium

Description | Editorial | Solutions | Submissions

## 876. Middle of the Linked List

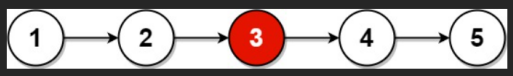
Solved

Easy | Topics | Companies

Given the `head` of a singly linked list, return *the middle node of the linked list.*

If there are two middle nodes, return **the second middle** node.

**Example 1:**




```
Input: head = [1,2,3,4,5]
Output: [3,4,5]
Explanation: The middle node of the list is node 3.
```

**Example 2:**

13.3K | 264 | 94 Online

Code

Java | Auto

```java
class Solution {
    public ListNode middleNode(ListNode head) {
        ListNode slow = head ;
        ListNode fast = head;
        while(fast != null && fast.next != null){
            slow = slow.next;
            fast = fast.next.next;
        }
        return slow;
    }
```

Saved | Ln 1, Col 1

Testcase | Test Result | Accepted

All Submissions

Accepted 36 / 36 testcases passed

rutuja_likhar2003 submitted at Apr 20, 2026 21:51

Analysis | Solution

Runtime: 0 ms | Beats 100.00%

Memory: 43.07 MB | Beats 19.86%

Problem List

Submit

02:25:12

Premium

Description | Editorial | Solutions | Submissions

# 2095. Delete the Middle Node of a Linked List

Solved

Medium | Topics | Companies | Hint

You are given the `head` of a linked list. **Delete** the **middle node**, and return *the* `head` *of the modified linked list.*

The **middle node** of a linked list of size `n` is the `⌊n / 2⌋th` node from the **start** using **0-based indexing**, where `⌊x⌋` denotes the largest integer less than or equal to `x`.

- For `n` = `1`, `2`, `3`, `4`, and `5`, the middle nodes are `0`, `1`, `1`, `2`, and `2`, respectively.

**Example 1:**

1 → 3 → 4 → 7 → 1 → 2 → 6

5K | 132 | 35 Online

Code

Java | Auto

```java
class Solution {
    public ListNode deleteMiddle(ListNode head) {
        if(head == null || head.next == null){
            return null;
        }
        ListNode slow = head;
        ListNode fast = head;
        ListNode prev = null;
        while(fast != null && fast.next!=null){
            prev = slow;
            slow = slow.next;
            fast = fast.next.next;

        }
        prev.next = slow.next;
```

Saved

Ln 13, Col 43

Testcase | Test Result | Accepted

All Submissions

Accepted 70 / 70 testcases passed

rutuja_likhar2003 submitted at Apr 20, 2026 22:23

Analysis | Solution